pfi

Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

Analyzing Text, Part 1

WEEK 4, DAY 2, PART 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Text is data too

Today's text: FOMC statements, one row for each statement

46

statements, one row each

2021-01-27

date of the first statement

2026-09-16

date of the last statement

The three columns of the file

Column	What it holds
`date`	the date of the statement, as text
`url`	the address of the statement's page
`text`	the statement as published, paragraphs separated by a line break

```
statements = pd.read_csv(data_file)
print(statements.shape)   # (46, 3)
```

The portfolio manager asks for a table

1. How long is each statement?
2. Which figures does it contain?
3. How often do a few chosen words appear?
4. How much wording does it share with the last?

Today we count words. We do not say what a statement means, and no tone is scored.

Reuse basis, quoted from the Board's website

https://www.federalreserve.gov/disclaimer.htm

"Unless otherwise indicated, information on Board's website is in the public domain and may be copied and distributed without permission. Please cite to the Board as the source of the information."

Source: Board of Governors of the Federal Reserve System, Federal Open Market Committee statements, federalreserve.gov.

Calendar page: https://www.federalreserve.gov/monetarypolicy/fomccalendars.htm. Retrieved on 2026-10-03. Text exactly as published.

Strings again: ten tools, ten Excel twins

Statement dated 2023-11-01, first paragraph: three sentences, stored as opening. For the Excel formulas it is in cell A2.

> Recent indicators suggest that economic activity expanded at a strong pace in the third quarter.
> Job gains have moderated since earlier in the year but remain strong, and the unemployment rate has remained low.
> Inflation remains elevated.

Job	Python	Excel	Result
Number of characters	`len(opening)`	`=LEN(A2)`	238
Lower case	`opening.lower()`	`=LOWER(A2)`	a new string
Capitals	`opening.upper()`	`=UPPER(A2)`	a new string
Is it there	`'inflation' in opening`	`=ISNUMBER(FIND("inflation", A2))`	True or False
Where it starts	`opening.find('Inflation')`	`=FIND("Inflation", A2)`	211 (Excel: 212)
A slice	`opening[:6]`	`=LEFT(A2, 6)`	Recent
How many times	`opening.count('rate')`	`=(LEN(A2) - LEN(SUBSTITUTE(A2, "rate", ""))) / LEN("rate")`	2
Cut into a list	`opening.split()`	`=TEXTSPLIT(A2, " ")`	37 words
Glue a list together	`' '.join(words[:5])`	`=TEXTJOIN(" ", TRUE, B2:F2)`	one string
Spaces off the ends	`piece.strip()`	`=TRIM(B2)`	no end spaces
Swap text	`opening.replace('.', '')`	`=SUBSTITUTE(A2, ".", "")`	3 full stops gone

Python counts positions from 0 and Excel from 1: find() returns 211 and FIND returns 212. TEXTSPLIT is in Microsoft 365 and Excel 2024.

A regular expression, piece by piece

```
range_pattern = r'[\d/-]+ to [\d/-]+ percent'
```

`[\d/-]+`	`to`	`[\d/-]+`	`percent`
the first figure: digits, slashes, and hyphens	a space, the letters t and o, a space	the second figure	a space and the word
matched `5-1/4`	matched `to`	matched `5-1/2`	matched `percent`

Inside the first piece

Piece	Meaning
`[ ]`	one character from the list inside
`\d`	any one digit
`/`	a slash
`-`	a plain hyphen, placed last
`+`	one or more of what is before it

Statement dated 2023-11-01, third paragraph, second sentence, as published:

In support of these goals, the Committee decided to maintain the target range for the federal funds rate at 5-1/4 to 5-1/2 percent.

```
re.findall(range_pattern, policy)
# ['5-1/4 to 5-1/2 percent']
match = re.search(r'([\d/-]+) to ([\d/-]+) percent', policy)
match.group(1), match.group(2), match.start()   # 5-1/4, 5-1/2, 218
re.sub(range_pattern, '[RANGE]', sentence)   # ... rate at [RANGE].
```

The same in Excel for Microsoft 365

```
=REGEXEXTRACT(A2, "[\d/-]+ to [\d/-]+ percent")
```

returns 5-1/4 to 5-1/2 percent

The sentence is in A2. Not in earlier versions.

Regular expressions: the pieces

Piece	Meaning	A pattern from the notebook	What that pattern describes
`\d`	any one digit	`\d+ percent`	digits, a space, and the word percent
`\w`	one word character: a letter, a digit, or an underscore	`^\w+`	the first word of the text
`\s`	one white space character: a space, a tab, or a line break	`\s+`	a run of white space
`\b`	the edge of a word: a position, not a character	`\brate\b`	rate as a whole word
`+`	one or more of whatever is just before it	`[\d/-]+`	a figure such as 5-1/4
`*`	none or more of whatever is just before it	`\w*rate\w*`	rate with any letters attached
`?`	none or one, which makes a piece optional	`(?:[A-Z]\. )?`	a middle initial, or none
`{4}`	exactly 4 of whatever is just before it	`^\d{4}-\d{2}-\d{2}$`	the shape of a date: 2023-11-01
`^ $`	anchors: the start and the end of the text or of a line	`\w+\.$`	a word and a full stop, ending a line
`( )`	a group: a part of the match, handed back by itself	`([\d/-]+) to ([\d/-]+) percent`	the two ends: 5-1/4 and 5-1/2
`[ ]`	one character from a list. [^] is one not on the list	`[^a-z\s]`	not a lower-case letter or white space
`.`	any one character. \. is a real full stop	`.{12}`	any twelve characters

Write every pattern as r'...', a raw string: a backslash is only a backslash. re.search() returns None when nothing matches.

Excel for Microsoft 365: `=REGEXTEST(A2, pattern)` `=REGEXEXTRACT(A2, pattern)` `=REGEXREPLACE(A2, pattern, new)` test, pull out, replace

Every version of Excel: wildcards in COUNTIF, SUMIF, SEARCH and the Find box. * is any run of characters, ? is any one. Flash Fill: Ctrl+E.

Too much, too little, and the fix

```
re.findall(r'\d+ percent', policy)
# ['2 percent', '2 percent', '2 percent', '2 percent']
re.findall(r'[\d/-]+ percent', policy)
# ['2 percent', '5-1/2 percent', '2 percent', '2 percent']
```

Matched too much, then fixed

4 matches, and the paragraph has 2 percent 3 times.

The fourth is the tail of 5-1/2 percent.

With [\d/-]+ each figure is whole: 3 read 2 percent.

```
statement.count('rate')   # 4
re.findall(r'\w*rate\w*', statement)
# ['moderated', 'rate', 'rate', 'rate']
re.findall(r'\brate\b', statement)
# ['rate', 'rate', 'rate']
```

Whole words: \b is the edge of a word

count() finds the four letters, inside moderated too.

With \b on each side, the word rate appears 3 times

in the statement dated 2023-11-01.

```
voting.count(';') + 1   # 12 names, counted by semicolons
re.findall(r'[A-Z][a-z]+ [A-Z]\. [A-Z][a-z]+', voting)   # 10
re.findall(r'[A-Z][a-z]+ (?:[A-Z]\. )?[A-Z][a-z]+', voting)   # 13
```

Matched too little, then too much

A middle initial required: 10 of the 12 names match.

Initial optional: 13. One match is a title, not a name.

Without the title: 12, the count by semicolons.

Print what a pattern matches before trusting how many. Test it against a count from somewhere else.

In Excel `=COUNTIF(B2:B38, "*rate*")` returns 2: moderated and rate. `=COUNTIF(B2:B38, "rate")` returns 1. The 37 words are in B2:B38.

Patterns for a desk: descriptions and CUSIPs

```
description_pattern = r'([A-Z]+)\s+(\d+(?:\.\d+)?)\s+(\d{2}/\d{2}/\d{4})'
```

Below each piece: its match in KRIX 7.875 08/26/2032

`([A-Z]+)`	`\s+`	`(\d+(?:\.\d+)?)`	`\s+`	`(\d{2}/\d{2}/\d{4})`
group 1, the ticker: one or more capital letters `KRIX`	one or more spaces `a space`	group 2, the coupon: digits, and an optional decimal part `7.875`	one or more spaces `a space`	group 3, the maturity: two digits, a slash, two digits, a slash, four digits `08/26/2032`

```
first_pattern = r'([A-Z]+) (\d+\.\d+) (\d{2}/\d{2}/\d{4})'
# Read: 6 of 8
# Not read: ['BEZE 7 06/18/2032', 'OSTV  6.5 04/01/2033']
assert match is not None, f'cannot read this description: {text}'
# Read: 8 of 8   mean coupon: 6.609 percent   latest maturity: 2055-09-03
```

Matched too little: 6 of 8, then 8 of 8

BEZE 7 has a coupon with no decimal point.

The OSTV line has two spaces after the ticker.

With \s+ and the optional (?:\.\d+)? all 8 are read.

Job	Python	Result in the notebook	Excel
Greedy	`re.search(r' at (.+) percent', policy)`	858 of the 944 characters	
Lazy	`re.search(r' at (.+?) percent', policy)`	13 characters: the rate of 2	`=TEXTBEFORE(TEXTAFTER(A2, " at "), " percent")`
A whole value	`re.fullmatch(r'\d{4}-\d{2}-\d{2}', value)`	False for a date plus a line break	`=REGEXTEST(A2, "^\d{4}-\d{2}-\d{2}$")`
Shape of a CUSIP	`cusips.str.fullmatch(r'[0-9A-Z]{9}')`	200 of 200. 4 damaged copies fail.	`=REGEXTEST(B2, "^[0-9A-Z]{9}$")`
A description	`re.fullmatch(description_pattern, text)`	8 of 8, as numbers and dates	`=TEXTSPLIT(TRIM(A2), " ")  =VALUE(C2)`

TEXTBEFORE, TEXTAFTER, TEXTSPLIT: Microsoft 365 and Excel 2024. REGEXTEST, and =REGEXEXTRACT(A2, pattern, 2) for the groups: Excel for Microsoft 365.

Every version of Excel: =LEN(B2)=9 and =EXACT(B2, UPPER(B2)). The eight descriptions and the 200 CUSIPs are invented, from the synthetic holdings file.

The AI check

Every notebook ends with code written in the style of an AI assistant. It has one bug. Find it.

1

Read

Read the code before you run it.

2

Predict

Decide what it should produce.

3

Run

Run it and look at the output.

4

Check

Compare with a number you worked out yourself.

Code that runs is not the same as code that is right.

What you can do now

- Measure, search, slice, split, join and replace text with string methods
- Write a regular expression from its pieces, and handle a search that finds nothing
- Test a pattern against a count, and fix one that matches too much or too little
- Tell greedy from lazy, and test a whole value with re.fullmatch()
- Read a bond description into a table, and test the shape of a CUSIP

Next Day 2, Part 2: cleaning, stopwords, and counting 46 statements